

Proyecto Programado: NextShop

Universidad: Instituto Tecnológico de Costa Rica

Carrera: Ingeniería del Software

Curso: Web 2

Profesor: Carlos Arias Rodriguez

Nombre del Proyecto Programado: NextShop

Integrantes: Oscar Marín y Edgardo Mora

Fecha: Junio de 2026

Índice

1. Resumen ejecutivo
2. Introducción
3. Planteamiento del problema
4. Objetivos del proyecto
5. Alcance funcional
6. Marco conceptual aplicado
7. Arquitectura general de NextShop
8. Modelo de datos y persistencia
9. Gestión de productos y catálogo visual
10. Módulo de imágenes de productos
11. Interfaz de usuario con Thymeleaf
12. Gestión de sesiones, login y perfiles
13. Carrito de compras

14. Módulo administrativo

15. Validaciones y reglas de negocio

16. Pruebas y evidencias

17. Control de versiones y trabajo colaborativo

18. Conclusiones

19. Anexos

ANEXOS

- Anexo A - Control del documento
- Anexo B - Ficha técnica del proyecto
- Anexo C - Documentos de apoyo
- Anexo D - Evidencias visuales
- Anexo E - Datos del módulo de imágenes

1. Resumen ejecutivo

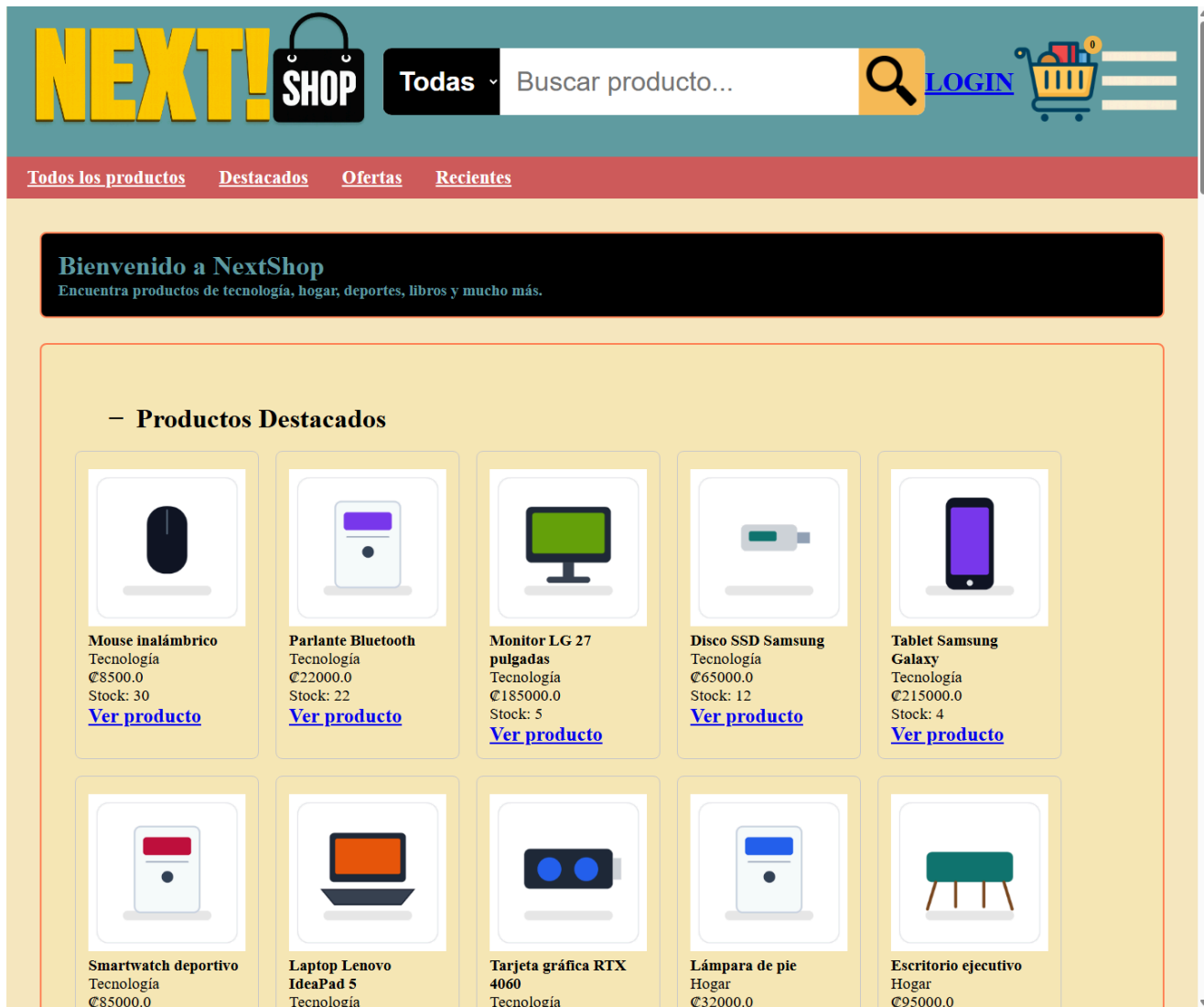
NextShop es una aplicación web desarrollada con Spring Boot y Thymeleaf para gestionar una tienda en línea. El sistema permite consultar productos, ver detalles, iniciar sesión, agregar productos al carrito, confirmar compras, consultar órdenes y administrar información del catálogo desde módulos internos.

La versión documentada trabaja con persistencia relacional en MySQL mediante Spring Data JPA. Esta implementación conserva la separación entre controladores, servicios, repositorios y vistas, y permite que productos, categorías, usuarios, inventario, órdenes y detalles de órdenes se administren desde base de datos.

Uno de los avances principales del proyecto fue la integración del catálogo visual de productos. Se auditaron 99 registros, se definieron 87 rutas PNG únicas y se conectó cada producto con su imagen mediante `imagePath`.

Esta implementación corresponde al uso del patrón MVC, recursos estáticos y separación por capas estudiados durante el curso.

Evidencia principal



La captura muestra la página principal renderizando productos reales del repositorio con imágenes cargadas desde imagePath. Evidencia la integración entre repositorio, servicio, controlador, vista Thymeleaf y recursos estáticos.

2. Introducción

NextShop fue desarrollado como una solución web para la gestión de una tienda en línea. La aplicación integra navegación pública, autenticación de usuarios, carrito de compras y módulos administrativos.

La implementación se organizó en capas para mantener responsabilidades claras. Los controladores atienden solicitudes HTTP, los servicios aplican reglas de negocio, los repositorios administran los datos de la aplicación y las vistas Thymeleaf presentan la información al usuario.

El proyecto también incorpora un módulo visual de productos. Las imágenes se almacenan como archivos PNG dentro de `static/images/products/` y se referencian desde cada producto mediante una ruta pública.

Evidencias relacionadas

- ``docs/evidencias/01-home-productos-imagenes.png``: muestra la página principal con productos e imágenes.
- ``docs/evidencias/02-detalle-producto-imagen.png``: muestra un producto individual con su imagen asociada.

3. Planteamiento del problema

Una tienda en línea necesita organizar productos, categorías, usuarios, inventario, carrito y recursos visuales de manera coherente. Si estas responsabilidades se mezclan, el sistema se vuelve difícil de mantener y de extender.

NextShop resuelve este problema mediante una estructura MVC por capas. La información de productos se administra desde repositorios JPA conectados a MySQL, la lógica se concentra en servicios y las vistas consumen los datos preparados por los controladores.

Durante la integración del catálogo visual se reemplazaron imágenes genéricas por imágenes específicas de producto sin modificar la base funcional del sistema. Para ello se documentó el catálogo, se prepararon imágenes PNG y se conectaron mediante `imagePath`.

Esta solución aplica separación de responsabilidades, renderizado dinámico y uso de recursos estáticos.

4. Objetivos del proyecto

Objetivo general

Desarrollar una aplicación web de comercio electrónico llamada NextShop que permita gestionar y visualizar productos, usuarios, carrito de compras y recursos estáticos aplicando los conceptos de Web 2.

Objetivos específicos

- Implementar una arquitectura MVC organizada por capas.
- Construir vistas dinámicas con Thymeleaf.

- Administrar datos mediante repositorios JPA conectados a MySQL.
- Permitir consulta, búsqueda y detalle de productos.
- Gestionar sesiones de usuario para clientes y administradores.
- Integrar imágenes de productos mediante rutas ``imagePath``.
- Validar el funcionamiento con compilación, rutas HTTP y evidencias visuales.

Evidencias relacionadas

- ``docs/evidencias/08-build-success.png``: confirma compilación exitosa.
 - ``docs/evidencias/09-git-status.png``: confirma el estado del repositorio al momento de generar evidencias.
-

5. Alcance funcional

La versión actual de NextShop incluye funcionalidades públicas, autenticadas y administrativas.

Funcionalidades públicas

- Visualización de productos en la página principal.
- Búsqueda de productos por categoría y nombre.
- Visualización del detalle de un producto.
- Registro e inicio de sesión.

Funcionalidades autenticadas

- Acceso al carrito de compras.
- Agregado de productos al carrito.
- Visualización de productos agregados.
- Confirmación de compra con generación de orden.
- Consulta de detalle de órdenes del cliente.
- Consulta de cuenta de usuario.

Funcionalidades administrativas

- Acceso al panel administrativo.
- Listado y búsqueda de productos.
- Creación y edición de productos.
- Activación y desactivación de productos.
- Gestión de clientes.
- Gestión de categorías.
- Gestión de inventario y stock mínimo.

Esta sección delimita el alcance entregado y resume las funcionalidades implementadas en la versión final del Proyecto Programado.

6. Marco conceptual aplicado

NextShop aplica los conceptos del curso desde una implementación concreta, no como un resumen teórico.

Arquitectura MVC

Los controladores reciben solicitudes HTTP, los servicios procesan reglas de negocio, los repositorios entregan datos y Thymeleaf renderiza las vistas.

Spring Boot

El proyecto utiliza Spring Boot para ejecutar la aplicación, registrar componentes y servir recursos web.

Thymeleaf

Las vistas usan expresiones como `th:src`, `th:text` y `th:href` para mostrar datos enviados por los controladores.

Persistencia con Spring Data JPA

Los datos principales se administran mediante repositorios JPA conectados a MySQL. Las interfaces de repositorio mantienen desacoplados los servicios de la implementación concreta de acceso a datos.

Recursos estáticos

Las imágenes de productos se sirven desde `src/main/resources/static/images/products/` mediante rutas públicas como `/images/products/laptop-lenovo-ideapad-5.png`.

Esta implementación corresponde al uso del patrón MVC, plantillas del lado servidor y recursos estáticos vistos durante el curso.

7. Arquitectura general de NextShop

La arquitectura del proyecto está organizada por capas:

Capa	Responsabilidad	Ejemplos
Controller	Atiende solicitudes HTTP y selecciona vistas o redirecciones	`HomeController`, `ProductDetailsController`, `ShoppingCartController`, `OrdersController`, `AdminInventoryController`, `AdminProductsController`
Service	Aplica reglas de negocio y validaciones	`ProductService`, `ClientService`, `ShoppingCartService`, `InventoryService`, `OrderService`
Repository	Accede a datos mediante interfaces y adaptadores JPA	`JpaProductRepositoryAdapter`, `JpaClientRepositoryAdapter`, `JpaCategoryRepositoryAdapter`, `JpaInventoryRepositoryAdapter`, `JpaOrderRepositoryAdapter`
Data / Model	Representa entidades del dominio	`Product`, `Client`, `Category`, `Inventory`, `Order`, `OrderItem`, `Cart`
Templates	Renderiza HTML con Thymeleaf	`home.html`, `ProductDetails.html`, `ShoppingCart.html`, `PayOrder.html`, `OrderDetails.html`, `AdminInventory.html`
Static	Expone CSS, imágenes y recursos públicos	`images/products/`, `css/`

Esta organización permite ubicar cada cambio en una capa específica. Los servicios trabajan contra interfaces de repositorio y no dependen directamente de la tecnología de almacenamiento. La implementación principal utiliza adaptadores JPA para acceder a MySQL, mientras que los repositorios en memoria quedan disponibles únicamente bajo el perfil in-memory.

El beneficio principal de esta arquitectura es la separación entre lógica de negocio y acceso a datos. Por ejemplo, la conexión de imágenes se mantuvo en el dato `imagePath` del producto y las vistas ya consumían `product.imagePath`, por lo que no fue necesario rediseñar la interfaz.

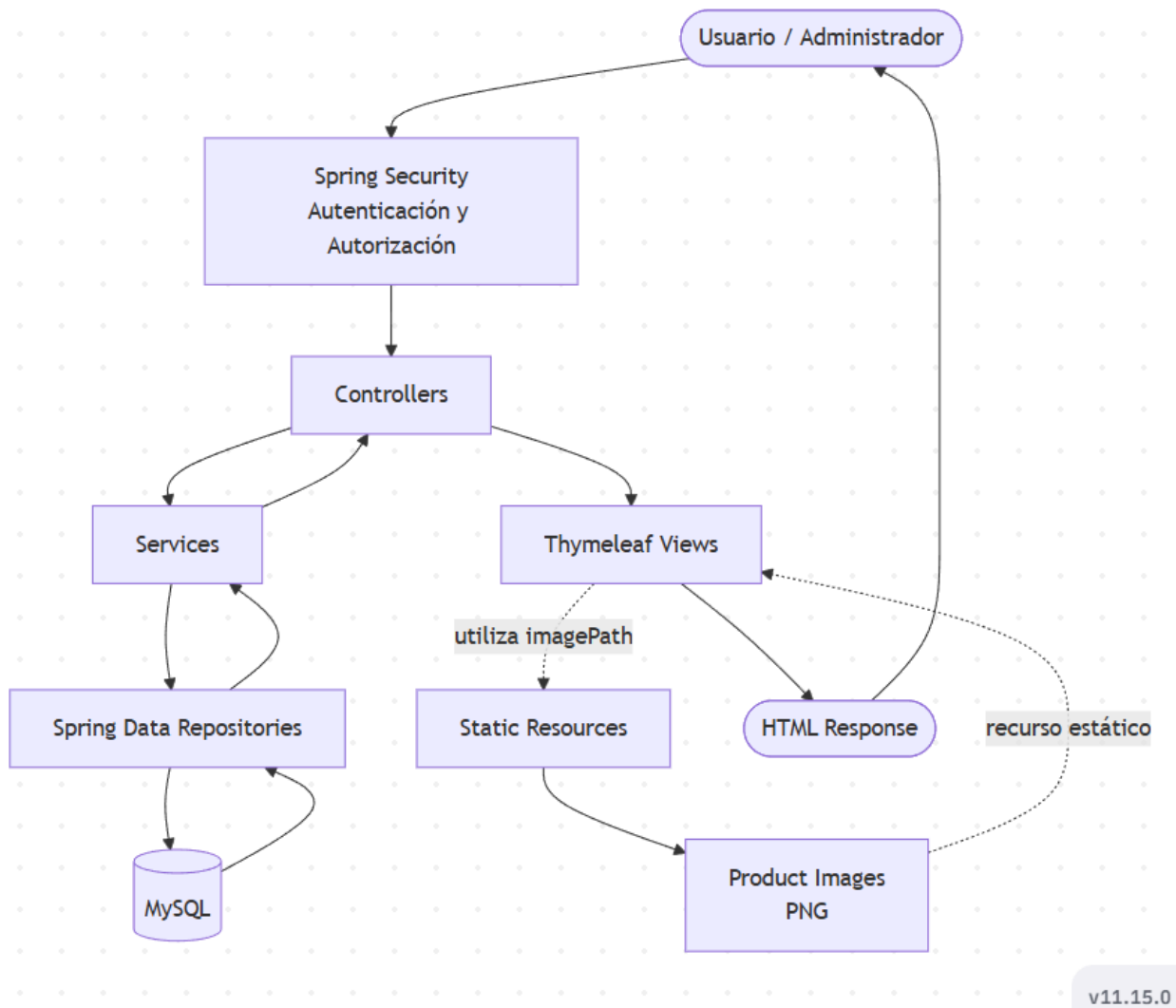


Figura 11. Diagrama de Componentes de la aplicación NextShop.

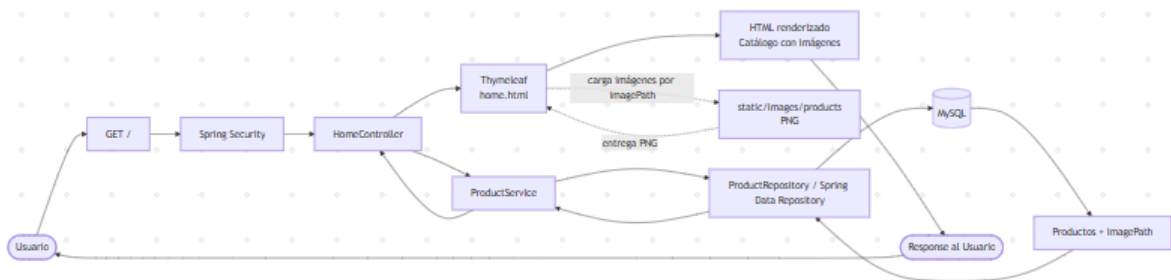


Figura 12. Flujo de atención de una solicitud en la aplicación NextShop utilizando la arquitectura Spring Boot.

La Figura 12 muestra el recorrido de una solicitud desde que el usuario realiza una petición hasta que la respuesta es presentada en el navegador. El controlador recibe la solicitud, delega el procesamiento en la capa de servicios y consulta la información mediante los repositorios conectados a la base de datos. Posteriormente, Thymeleaf genera la vista

utilizando los datos obtenidos y carga las imágenes correspondientes mediante el atributo `imagePath`, produciendo finalmente la respuesta HTML enviada al usuario.

Este flujo evidencia la separación de responsabilidades implementada en la aplicación y la interacción entre las diferentes capas que conforman la arquitectura de NextShop.

Evidencias relacionadas

- ``docs/evidencias/06-productrepository-imagepath.png``: muestra productos con rutas reales.
- ``docs/evidencias/07-carpeta-static-images-products.png``: muestra la carpeta de imágenes estáticas.

8. Modelo de datos y persistencia

NextShop utiliza MySQL como base de datos relacional para manejar productos, categorías, clientes, inventario, órdenes y detalles de órdenes. Se eligió MySQL por ser un motor relacional estable, ampliamente utilizado y adecuado para representar relaciones entre productos, categorías, usuarios, inventario y pedidos.

La integración se realiza con Spring Data JPA porque permite trabajar con entidades Java y repositorios declarativos, reduciendo código repetitivo de acceso a datos y manteniendo una capa de persistencia coherente con Spring Boot. El proyecto conserva una estructura por interfaces para separar los servicios de la tecnología de persistencia. Los repositorios en memoria permanecen como alternativa de perfil, mientras que la ejecución principal utiliza los adaptadores JPA y la base `nextshopdb`.

En el caso de productos, cada registro incluye información como identificador, SKU, nombre, descripción, precio, categoría, propiedades, estado, stock y ruta de imagen. El campo `imagePath` queda persistido como metadato del producto y apunta a archivos estáticos PNG.

Hibernate administra la creación y actualización de tablas mediante la configuración `spring.jpa.hibernate.ddl-auto=update`. Además, `DataInitializer` carga la información inicial cuando la base está vacía: categorías, clientes, productos e inventario. Este proceso permite levantar una base local funcional sin copiar manualmente archivos SQL ni respaldos de base de datos.

El carrito de compras se mantiene en sesión/memoria por decisión de diseño, ya que representa un estado temporal asociado al usuario autenticado durante su navegación.

Esta implementación corresponde al uso de repositorios, entidades, ORM y persistencia relacional vistos durante el curso.

8.1 Base de datos local con Docker Compose

El proyecto incluye `docker-compose.yml` para levantar una base MySQL local reproducible. El servicio utiliza una imagen compatible de MySQL, expone el puerto local 3307 hacia el puerto interno 3306 y crea la base `nextshopdb` al iniciar el contenedor por primera vez.

El archivo no depende de rutas absolutas del equipo de un desarrollador. Utiliza un volumen nombrado de Docker para conservar los datos locales sin versionar la base física en Git. Su propósito es que cada integrante pueda levantar su propio ambiente local con la misma configuración general de base de datos.

9. Gestión de productos y catálogo visual

El catálogo visual se construyó mediante una auditoría documental del inventario existente. El objetivo fue identificar los productos reales usados por el sistema y definir una imagen específica para cada producto único.

Documentos de apoyo:

- ``docs/inventario-productos-imagenes.md``
- ``docs/catalogo-visual-productos.md``
- ``docs/fuente-imagenes-productos.md``
- ``docs/catalogo-imagenes-productos.md``

Resultados del catálogo:

Indicador	Resultado
Registros auditados	99
Rutas PNG únicas	87
Duplicados controlados	12
Imágenes faltantes	0
Fallback conservado	<code>`productlcon.png`</code>

Los productos duplicados conservan la misma imagen cuando representan el mismo artículo. Esto evita inconsistencias visuales y mantiene trazabilidad con los registros originales.

Evidencias relacionadas

- `docs/evidencias/01-home-productos-imagenes.png`: muestra productos renderizados con imágenes.
- `docs/evidencias/07-carpeta-static-images-products.png`: muestra los PNG disponibles.

10. Módulo de imágenes de productos

El módulo de imágenes conecta cada producto con una ruta pública mediante el atributo `imagePath`. Las imágenes se encuentran en:

```
src/main/resources/static/images/products/
```

La ruta pública utilizada por la aplicación sigue este formato:

```
/images/products/nombre-del-producto.png
```

La integración se realizó conectando cada producto con su ruta `imagePath` y conservando esos valores en la persistencia de productos. No se modificaron nombres, categorías, precios, descripciones ni lógica de negocio. El fallback `productIcon.png` se conserva como imagen predeterminada del sistema, pero los productos existentes ya usan rutas específicas.

Esta implementación evidencia el uso de recursos estáticos en Spring Boot y renderizado dinámico con Thymeleaf.

Evidencias



La captura muestra una imagen servida directamente desde `/images/products/`. Confirma que Spring Boot expone correctamente los archivos ubicados en `static/images/products/`.

ProductRepository.java - imagePath reales

```

60:         toys = categoryRepository.findByCategoryName("Juguetes").orElseThrow();
61:         others = categoryRepository.findByCategoryName("Otros").orElseThrow();
62:
63:         // -----
64:         // TECNOLOGÍA - NORMALES
65:         // -----
66:         this.productList.add(new Product(1L, "USB-001", "Memoria USB 64 GB", "Dispositivo portátil de almacenamiento.", 6500, "/images/technology, null, true));
67:         this.productList.add(new Product(2L, "CAM-001", "Webcam HD", "Cámara web para videollamadas.", 14500, "/images/products/webcam.png", technology, null, true));
68:         this.productList.add(new Product(3L, "CHR-001", "Cargador USB", "Cargador rápido para dispositivos móviles.", 9500, "/images/technology, null, true));
69:         this.productList.add(new Product(4L, "STD-001", "Base para monitor", "Soporte para monitor de escritorio.", 12500, "/images/technology, null, true));
70:         this.productList.add(new Product(5L, "PRN-001", "Impresora multifuncional", "Impresora para hogar y oficina.", 125000, "/images/technology, Arrays.asList(new Property("Conectividad", "WiFi"), new Property("Tecnología", "Inyección de tinta")), true));
71:         this.productList.add(new Product(6L, "RTR-001", "Router WiFi 6", "Router de alta velocidad.", 58000, "/images/products/router.png", technology, Arrays.asList(new Property("Velocidad", "1800 Mbps"), new Property("Bandas", "Dual Band")), true));
72:
73:         // -----
74:         // TECNOLOGÍA - DESTACADOS
75:         // -----
76:         this.productList.add(new Product(7L, "MOU-001", "Mouse inalámbrico", "Mouse óptico para uso diario.", 8500, "/images/products/mouse.png", technology, null, true, 0, LocalDateTime.now().minusDays(90), true));
77:         this.productList.add(new Product(8L, "SPK-001", "Parlante Bluetooth", "Parlante inalámbrico portátil.", 22000, "/images/products/speaker.png", technology, null, true, 0, LocalDateTime.now().minusDays(60), true));
78:         this.productList.add(new Product(9L, "MON-001", "Monitor LG 27 pulgadas", "Monitor para productividad.", 185000, "/images/products/monitor.png", technology, Arrays.asList(new Property("Resolución", "2560x1440"), new Property("Frecuencia", "144 Hz")), true, 0, LocalDateTime.now().minusDays(120), true));
79:         this.productList.add(new Product(10L, "SSD-001", "Disco SSD Samsung", "Unidad de almacenamiento.", 65000, "/images/products/ssd.png", technology, Arrays.asList(new Property("Capacidad", "1 TB"), new Property("Formato", "M.2 NVMe")), true, 0, LocalDateTime.now().minusDays(12), true));
80:         this.productList.add(new Product(11L, "TAB-001", "Tablet Samsung Galaxy", "Tablet para entretenimiento.", 215000, "/images/products/tablet.png", technology, Arrays.asList(new Property("Pantalla", "10.4 pulgadas"), new Property("Almacenamiento", "128 GB")), true, 0, LocalDateTime.now().minusDays(15), true));
81:         this.productList.add(new Product(12L, "SMW-001", "Smartwatch deportivo", "Reloj inteligente.", 85000, "/images/products/smartwatch.png", technology, Arrays.asList(new Property("Pantalla", "AMOLED"), new Property("Resistencia", "5 ATM")), true, 0, LocalDateTime.now().minusDays(5), true));
82:
83:         // -----
84:         // TECNOLOGÍA - OFERTAS
85:         // -----
86:         this.productList.add(new Product(13L, "KEY-001", "Teclado mecánico", "Teclado mecánico para oficina y gaming.", 28500, "/images/products/keyboard.png", technology, null, false, 15, LocalDateTime.now().minusDays(45), true));
87:         this.productList.add(new Product(14L, "AUD-001", "Auriculares inalámbricos", "Auriculares Bluetooth de uso diario.", 18000, "/images/products/headphones.png", technology, null, false, 20, LocalDateTime.now().minusDays(25), true));
88:
89:         // -----
90:         // TECNOLOGÍA - DESTACADOS + OFERTA
91:         // -----

```

La captura muestra productos con rutas como /images/products/laptop-lenovo-ideapad-5.png. Evidencia que el repositorio conecta datos del producto con recursos estáticos.

src/main/resources/static/images/products/

```

aspiradora-vertical.png
audifonos-inalambricos.png
balon-de-baloncesto.png
balon-de-futbol.png
banco-para-pesas.png
base-para-monitor.png
bicicleta-de-montana.png
bloques-de-construccion.png
boligrafo-azul.png
boligrafo-negro.png
bolso-de-mano.png
botella-deportiva.png
cafetera-domestica.png
calcetines-deportivos.png
calculadora-cientifica.png
calculadora-financiera.png
caminadora-electrica.png
camiseta-casual.png
canasta-de-ropa.png
cargador-usb-rapido.png
carpeta-plastica.png
carro-control-remoto.png
carro-de-juguete.png
casa-de-munecas.png
chaqueta-impermeable.png
chaqueta-ligera.png
cinta-adhesiva.png
cinturon-de-cuero.png
clean-code.png
colchon-ortopedico.png
cuaderno-universitario.png
cuerda-para-saltar.png
disco-ssd-samsung.png
don-quijote-de-la-mancha.png
drone-recreativo.png
dune-libro.png
el-hobbit.png
el-senor-de-los-anillos.png
enciclopedia-universal.png
escritorio-ejecutivo.png
fundacion-libro.png
gorra-ajustable.png
harry-potter-piedra-filosofal.png
impresora-multifuncional.png
introduccion-a-java.png
jeans-clasicos.png
juego-de-comedor.png
juego-de-cuchillos.png
juego-de-mesa.png
lampara-de-pie.png
laptop-lenovo-ideapad-5.png
lego-city.png
libro-1984.png
licuadora-domestica.png
mancuernas-ajustables.png
marcadores-de-colores.png
mat-para-yoga.png
memoria-usb-64gb.png
microondas-domestico.png
mochila-ejecutiva.png
monitor-lg-27-pulgadas.png
mouse-inalambrico.png
muneca.png
organizador-de-escritorio.png

```

La captura muestra los archivos PNG disponibles, incluyendo productIcon.png. Evidencia que las rutas definidas apuntan a archivos físicos existentes.

11. Interfaz de usuario con Thymeleaf

Las vistas de NextShop se renderizan con Thymeleaf. Los controladores cargan datos en el `Model` y las plantillas los muestran mediante expresiones dinámicas.

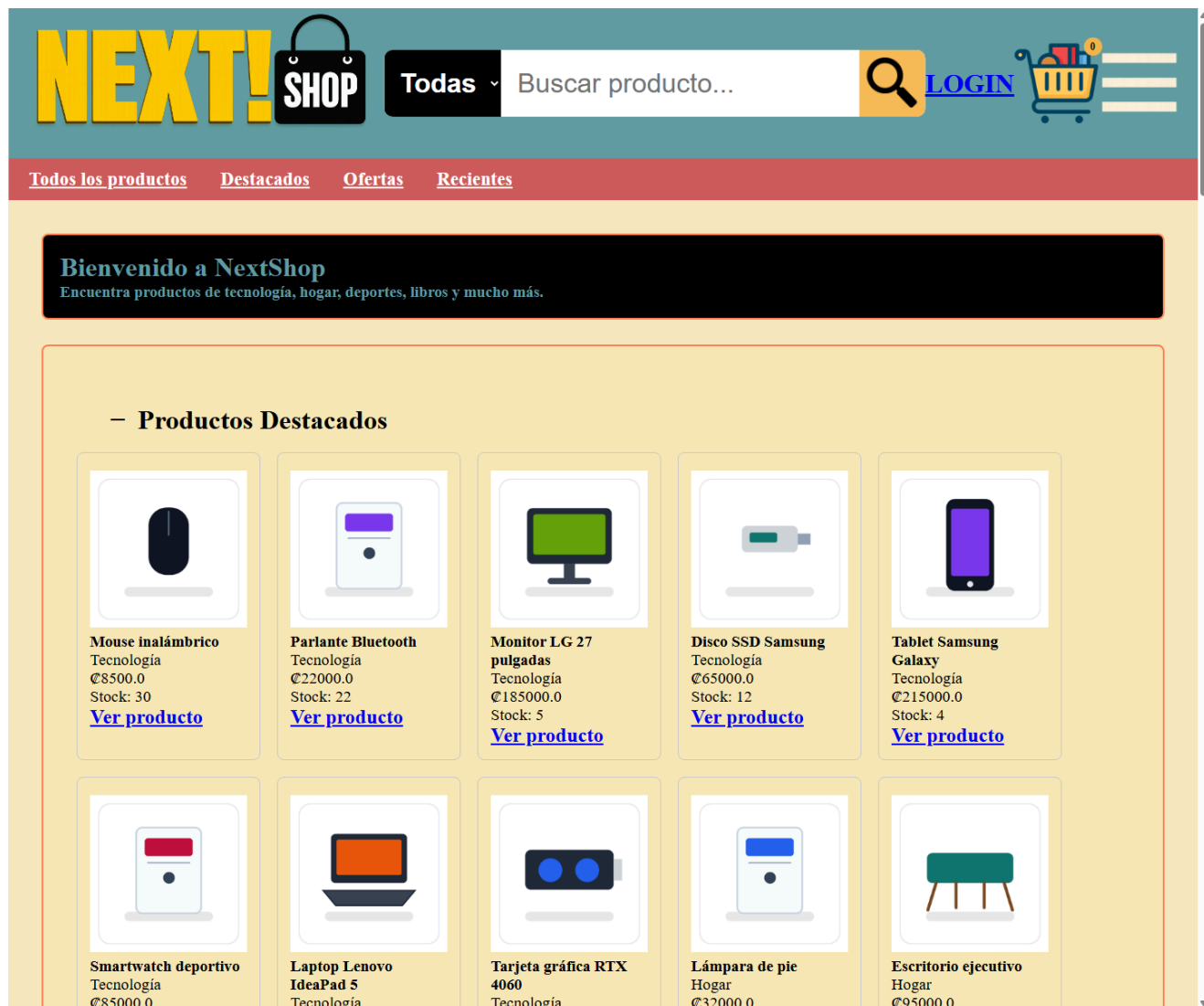
En el catálogo y detalle de productos, las imágenes se muestran usando `product.imagePath`. Esto permite que la vista no conozca el nombre físico de cada archivo; solo consume la ruta definida en el producto.

Vistas relacionadas:

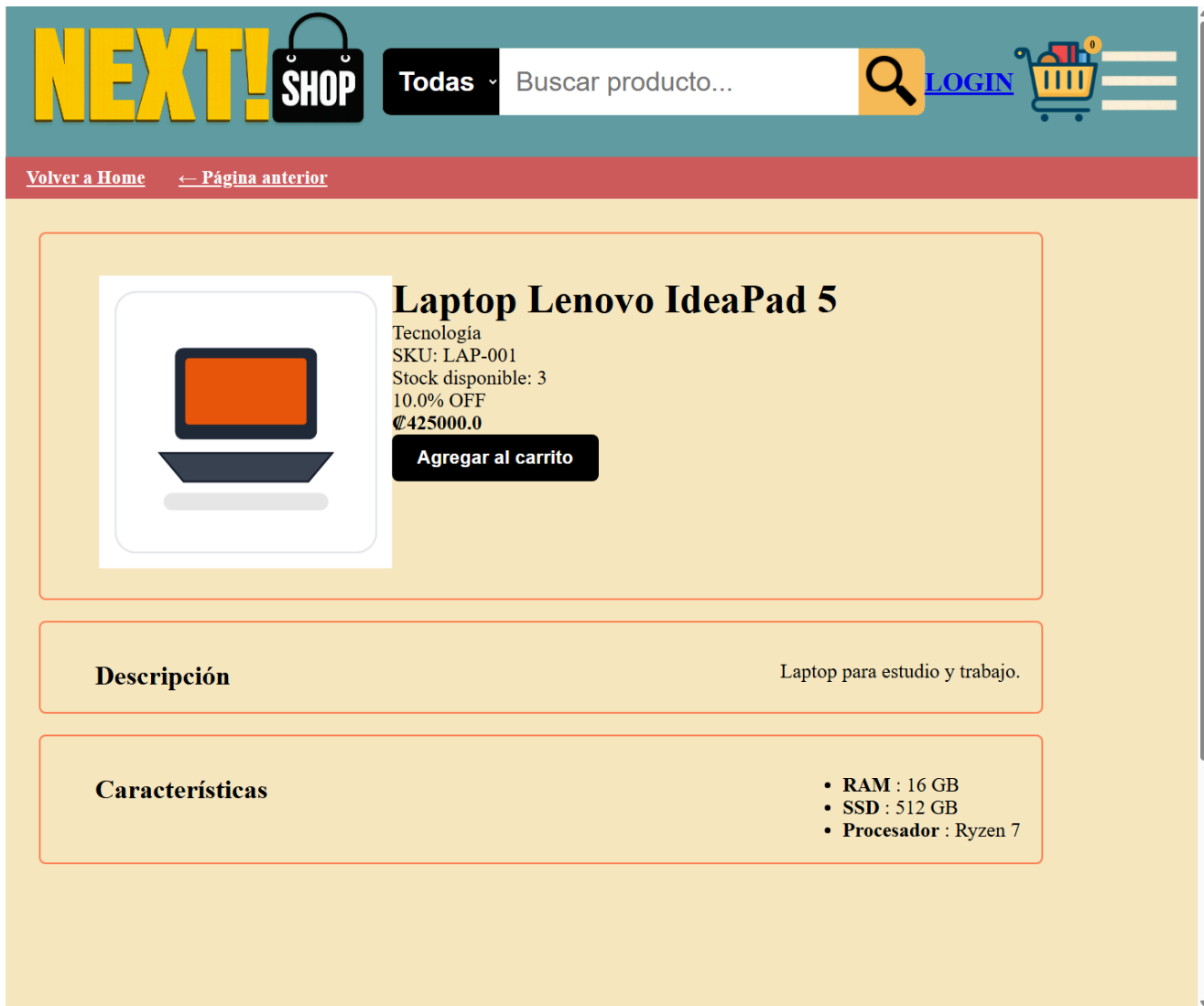
- ``src/main/resources/templates/pages/home.html``
- ``src/main/resources/templates/pages/ProductDetails.html``
- ``src/main/resources/templates/pages/ShoppingCart.html``
- ``src/main/resources/templates/pages/PayOrder.html``
- ``src/main/resources/templates/pages/OrderDetails.html``
- ``src/main/resources/templates/fragments/ProductData.html``
- ``src/main/resources/templates/managment/AdminProducts.html``
- ``src/main/resources/templates/managment/AdminInventory.html``

Esta implementación corresponde al renderizado de vistas del lado servidor visto en el curso.

Evidencias



La captura muestra tarjetas de producto con nombre, categoría, precio, stock e imagen. Evidencia el uso de Thymeleaf para presentar datos del backend.



La captura muestra el detalle de un producto individual con su imagen asociada. Evidencia la relación entre ruta dinámica y vista de detalle.

12. Gestión de sesiones, login y perfiles

NextShop maneja usuarios mediante sesión HTTP. El sistema diferencia entre clientes y administradores, y utiliza esa información para permitir o restringir funcionalidades.

El login valida correo, contraseña y estado activo del usuario. Una vez autenticado, el cliente queda disponible en sesión para operaciones como acceder al carrito o agregar productos.

Esta implementación corresponde al manejo de estado con HttpSession, un concepto fundamental en aplicaciones web tradicionales.

Evidencia relacionada

- ``docs/evidencias/04-carrito-imagen-producto.png``: muestra un producto agregado al carrito en una sesión de cliente.
-

13. Carrito de compras

El carrito de compras está asociado al cliente autenticado. Para agregar un producto, el sistema valida que el usuario tenga sesión activa, que el producto exista y que cuente con stock disponible.

La funcionalidad se implementa principalmente con:

- ``ShoppingCartController``
- ``ShoppingCartService``
- ``ShoppingCartRepository``
- ``InventoryService``
- ``ProductService``
- ``OrdersController``
- ``OrderService``

El carrito muestra producto, cantidad, stock, subtotal y resumen del pedido. También utiliza `item.product.imagePath` para presentar la imagen del producto agregado. Durante la confirmación de compra, el sistema valida inventario, registra la orden, descuenta cantidades disponibles y permite consultar el detalle de la orden generada.

Esta implementación corresponde al uso de formularios, solicitudes POST, redirecciones y sesión HTTP.

Evidencia

Carrito de Compras



Laptop Lenovo IdeaPad 5

Tecnologia

€ 425000.0

Cantidad: 1

Stock: 3

Subtotal: € 425000.0

-

1

+

Eliminar

Resumen del Pedido

Artículos: 1

Subtotal: € 425000.0

Descuentos: -€ 42500.0

Impuesto: € 55250.0

Envío: € 0.0

La captura muestra un producto agregado al carrito con su imagen. Evidencia la relación entre sesión de cliente, carrito e imagen del producto.

14. Módulo administrativo

El módulo administrativo permite consultar y mantener información del sistema. Incluye rutas como:

- `/AdminProducts`
- `/AdminClients`
- `/AdminCategories`
- `/AdminInventory`
- `/AdminDashboard`

Desde estas pantallas se gestionan productos, clientes, categorías e inventario según las opciones implementadas. Las operaciones incluyen búsqueda, edición, activación,

desactivación y actualización de cantidades disponibles y stock mínimo.

El acceso administrativo depende del perfil del usuario. Por esta razón, el informe documenta el módulo a partir de su implementación, sus rutas y las evidencias de control de versiones, sin alterar seguridad ni vistas para generar una captura artificial.

Esta implementación corresponde al control de acceso básico por perfil y al mantenimiento de registros desde formularios web.

Evidencia relacionada

- `docs/evidencias/README.md`: contiene el índice de evidencias visuales generadas para el informe.

15. Validaciones y reglas de negocio

NextShop aplica reglas de negocio desde controladores y servicios. Las principales son:

- No registrar clientes con correo duplicado.
- Validar correo, contraseña y estado del usuario al iniciar sesión.
- Diferenciar perfil cliente y perfil administrador.
- Evitar SKU duplicado al crear productos.
- Permitir activar o desactivar productos y usuarios.
- Validar existencia del producto antes de agregarlo al carrito.
- Validar stock disponible antes de agregar al carrito.
- Validar cantidades antes de actualizar inventario.
- Descontar inventario al confirmar una orden.
- Registrar órdenes y sus detalles asociados al cliente.
- Asociar el carrito al cliente autenticado.

Estas reglas reducen errores de uso y mantienen coherencia entre interfaz, datos y lógica.

Esta implementación corresponde a la validación del lado servidor estudiada en el curso.

16. Pruebas y evidencias

Las pruebas realizadas verifican que la aplicación compila, sirve recursos estáticos y renderiza productos con imágenes.

16.1 Pruebas funcionales

Prueba	Resultado	Evidencia
Home `/	Correcto	`01-home-productos-imagenes.png`
Detalle `/products/15`	Correcto	`02-detalle-producto-imagen.png`
Imagen estática	HTTP 200	`03-ruta-estatica-imagen.png`
Carrito con producto	Correcto	`04-carrito-imagen-producto.png`
Confirmación de orden	Correcto	Validado mediante flujo de compra
Detalle de orden	Correcto	Vista `OrderDetails.html`
Administración de inventario	Correcto	Vista `AdminInventory.html`
Login contra MySQL	Correcto	Validado con usuarios persistentes
Docker Compose operativo	Correcto	MySQL disponible en puerto local `3307`

16.2 Pruebas técnicas

Verificación	Resultado
Productos con `imagePath`	99
Rutas únicas usadas	87
Duplicados con imagen compartida	12
Imágenes faltantes	0
Fallback conservado	`productIcon.png`
Productos persistentes en MySQL	99
Inventarios persistentes en MySQL	99
Cientes iniciales persistentes en MySQL	3
Órdenes persistentes en MySQL	Correcto

Descuento de inventario por compra	Correcto
Catálogo visual mediante `imagePath`	Correcto

16.3 Compilación

El proyecto fue compilado con Maven:

```
mvn clean compile
```

Resultado esperado y obtenido:

```
BUILD SUCCESS
```

16.4 Evidencias visuales

Evidencia	Archivo	Qué demuestra
Home con productos	`docs/evidencias/01-home-productos-imagenes.png`	Renderizado del catálogo con imágenes
Detalle de producto	`docs/evidencias/02-detalle-producto-imagen.png`	Imagen asociada al producto seleccionado
Ruta estática	`docs/evidencias/03-ruta-estatica-imagen.png`	Spring Boot sirviendo PNG desde `static/`
Carrito	`docs/evidencias/04-carrito-imagen-producto.png`	Producto en carrito con imagen
ProductRepository	`docs/evidencias/06-productrepository-imagepath.png`	Rutas `imagePath` reales
Carpeta de imágenes	`docs/evidencias/07-carpeta-static-images-products.png`	Archivos PNG físicos disponibles
Build	`docs/evidencias/08-build-success.png`	Compilación exitosa
MySQL y Docker Compose	`docker-compose.yml`	Base local reproducible para persistencia
Git status	`docs/evidencias/09-git-status.png`	Estado del repositorio
Git log	`docs/evidencias/10-git-log.png`	Historial reciente de commits
Diagrama de componentes	`docs/evidencias/11-diagrama-componentes-nextshop.png`	Organización por capas y componentes

Flujo Request-Response	`docs/evidencias/12-diagrama 2.png`	Recorrido de una solicitud en Spring Boot
------------------------	-------------------------------------	---

Esta sección evidencia validación funcional, técnica y de control de versiones.

17. Control de versiones y trabajo colaborativo

El proyecto utiliza Git para registrar cambios, mantener trazabilidad y facilitar el trabajo colaborativo entre integrantes. La rama documentada es:

```
integracion-edgardo-nueva-version
```

Commits relevantes:

```
a3147d5 Integrar ultima actualizacion de Edgardo
813ec1b Agregar diagramas finales al informe
4b0a5eb Correcciones de bugs, estructura de pedidos, validaciones,
inventario e historial de órdenes
3a8ce9d Actualizar documentacion final del proyecto
690c794 Agregar compose local para MySQL
45e2dd1 Agregar persistencia JPA con MySQL
34839c2 Conectar imagenes de productos y corregir template home
```

Git versiona el código fuente, las entidades JPA, los adaptadores de repositorio, docker-compose.yml, DataInitializer, la configuración de la aplicación, la documentación y las imágenes del catálogo. La base de datos física no se versiona; cada desarrollador obtiene una base local propia al levantar Docker y ejecutar la aplicación.

El flujo colaborativo se ejecuta de la siguiente forma:

```
git pull
docker compose up -d
./mvnw spring-boot:run
```

En Windows también puede ejecutarse:

```
mvnw.cmd spring-boot:run
```

Con este flujo, Docker levanta MySQL, Hibernate crea o actualiza las tablas y DataInitializer carga automáticamente categorías, clientes, productos e inventario. Las imágenes ya están versionadas en Git y el campo imagePath queda persistido en MySQL. Como resultado, cada integrante obtiene una copia funcional del proyecto sin copiar manualmente bases de datos ni archivos SQL.

El uso de Git respalda el trabajo colaborativo y permite auditar los cambios realizados.

Evidencias

git status

```
On branch integracion-edgardo-imagenes
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    docs/evidencias/01-home-productos-imagenes.png
    docs/evidencias/02-detalle-producto-imagen.png
    docs/evidencias/03-ruta-estatica-imagen.png
    docs/evidencias/04-carrito-imagen-producto.png
    docs/evidencias/06-productrepository-imagepath.png
    docs/evidencias/07-carpeta-static-images-products.png
    docs/evidencias/08-build-success.png
    docs/evidencias/09-git-status.png
    docs/evidencias/10-git-log.png

nothing added to commit but untracked files present (use "git add" to track)
```

La captura muestra el estado del repositorio durante la generación de evidencias. Es importante para respaldar la trazabilidad del trabajo documentado.

git log --oneline -5

```
34839c2 Conectar imagenes de productos y corregir template home
8e79928 Merge branch 'backup-imagenes-nextshop' into integracion-edgardo-imagenes
e7ca14e Backup avances catalogo e imagenes de productos
71f0e67 Resolución de bugs menores, y corrección en consulta principal de home, que estaba permitiendo ver productos desac
f8f12aa Incorporacion de cambio, método, calculos de items en carrito, impuesto, total, etc
```

La captura muestra los commits recientes. Evidencia la trazabilidad del avance del módulo de imágenes y del fix de home.

18. Conclusiones

NextShop demuestra una aplicación web funcional construida con Spring Boot, Thymeleaf y una arquitectura MVC por capas. El proyecto integra navegación pública, detalle de productos, sesiones de usuario, carrito de compras, confirmación de órdenes, consulta de detalle de órdenes y módulos administrativos.

La persistencia relacional con MySQL, Spring Data JPA e Hibernate permite almacenar categorías, productos, inventario, clientes, órdenes y detalles de órdenes. El uso de Docker Compose facilita una base local reproducible, mientras que DataInitializer carga los datos iniciales necesarios para ejecutar el sistema.

La integración del catálogo visual fortalece la experiencia de usuario y evidencia el uso correcto de recursos estáticos. Cada producto del inventario queda conectado a una imagen mediante `imagePath`, manteniendo una separación clara entre datos persistidos y archivos físicos versionados.

El trabajo colaborativo se respalda con Git, que versiona el código fuente, la documentación, Docker Compose, las entidades JPA, los adaptadores de repositorio, las imágenes del catálogo y la configuración necesaria para reproducir el entorno. Esta implementación consolida los conceptos prácticos de Web 2: controladores, servicios, repositorios, vistas dinámicas, sesiones, validaciones, persistencia y control de versiones.

Resumen ejecutivo de funcionalidades

Funcionalidad	Estado
Login	Implementado
Catálogo	Implementado
CRUD Productos	Implementado
CRUD Categorías	Implementado
Gestión de Clientes	Implementado
Gestión de Inventario	Implementado
Carrito	Implementado
Confirmación de Orden	Implementado
Detalle de Orden	Implementado
Integración de imágenes	Implementado
Recursos estáticos	Implementado
Validaciones	Implementado
Build Maven	Correcto

19. Anexos

Anexo A: Control del documento

Elemento	Valor
Archivo principal	`docs/informe-final.md`
Archivo PDF	`docs/informe-final.pdf`

Evidencias visuales	`docs/evidencias/`
Documentos de apoyo	`docs/ProyectoProgramado.md`, `docs/inventario-productos-imagenes.md`, `docs/catalogo-visual-productos.md`, `docs/fuente-imagenes-productos.md`, `docs/catalogo-imagenes-productos.md`
Estado	Versión final para entrega

Anexo B: Ficha técnica del proyecto

Elemento	Valor
Proyecto	NextShop
Curso	Web 2
Profesor	Carlos Arias Rodriguez
Integrantes	Oscar Marín y Edgardo Mora
Tipo de sistema	Tienda en línea
Framework	Spring Boot
Motor de plantillas	Thymeleaf
Arquitectura	MVC por capas
Lenguaje principal	Java
Persistencia actual	MySQL con Spring Data JPA
Persistencia	Repositorios JPA sobre interfaces de acceso a datos
Base de datos local	MySQL `nextshopdb` mediante Docker Compose
Configuración local	`src/main/resources/application.properties`
Ejecución local	`docker compose up -d` y `mvnw.cmd spring-boot:run`
Recursos estáticos	CSS e imágenes desde `src/main/resources/static/`
Control de versiones	Git

Build	Maven
Estado	Funcional

Anexo C: Documentos de apoyo

- `docs/ProyectoProgramado.md`
- `docs/inventario-productos-imagenes.md`
- `docs/catalogo-visual-productos.md`
- `docs/fuente-imagenes-productos.md`
- `docs/catalogo-imagenes-productos.md`

Anexo D: Evidencias visuales

- `docs/evidencias/README.md`
- `docs/evidencias/01-home-productos-imagenes.png`
- `docs/evidencias/02-detalle-producto-imagen.png`
- `docs/evidencias/03-ruta-estatica-imagen.png`
- `docs/evidencias/04-carrito-imagen-producto.png`
- `docs/evidencias/06-productrepository-imagepath.png`
- `docs/evidencias/07-carpeta-static-images-products.png`
- `docs/evidencias/08-build-success.png`
- `docs/evidencias/09-git-status.png`
- `docs/evidencias/10-git-log.png`
- `docs/evidencias/11-diagrama-componentes-nextshop.png`
- `docs/evidencias/12-diagrama 2.png`

Anexo E: Datos del módulo de imágenes

Indicador	Valor
Registros de producto auditados	99
Rutas PNG únicas	87
Duplicados controlados	12
Imágenes faltantes	0
Fallback conservado	`productIcon.png`